

Rapport de projet RITAL
Analyse de sentiments et reconnaissance du locuteur

ARJDAL Younes 21203492

Table des matières

1 Introduction
2 Movie
    2.1 Analyse
    2.2 Expérimentation
        2.2.1 Protocole Bag of words
            Impact du prétraitement par modèle
            Comparaison des modèles par prétraitement
            Conclusion BoW
        2.2.2 Protocole TF-IDF
            Comparaison par prétraitement
            Conclusion TF-IDF
        2.2.3 Comparaison et tendance des modèles BoW vs TF-IDF :
            Configurations retenues pour la recherche d'hyperparamètres et le challenge :
        2.2.3 Résultats Challenge
3 Président
    3.1 Analyse
    3.2 Expérimentation
        3.2.1 Baseline
        3.2.2 Introduction à CamemBERT
        3.2.3 Résultats Challenge
4 Conclusion

1 Introduction

Ce projet s'inscrit dans le cadre de l'étude de méthodes de classification de texte en traitement automatique du langage naturel. Deux tâches distinctes sont abordées : une tâche d'analyse de sentiment sur des critiques de films (Movie), et une tâche d'identification de locuteur sur des discours présidentiels (Président).

Ces deux problèmes, bien que formulés de manière similaire, présentent des caractéristiques très différentes. Le premier repose sur un corpus équilibré et se prête bien à l'analyse de l'impact du prétraitement et des modèles classiques. Le second, en revanche, introduit des difficultés supplémentaires, notamment un fort déséquilibre de classes et une dépendance au contexte, rendant nécessaire l'utilisation de modèles plus avancés.

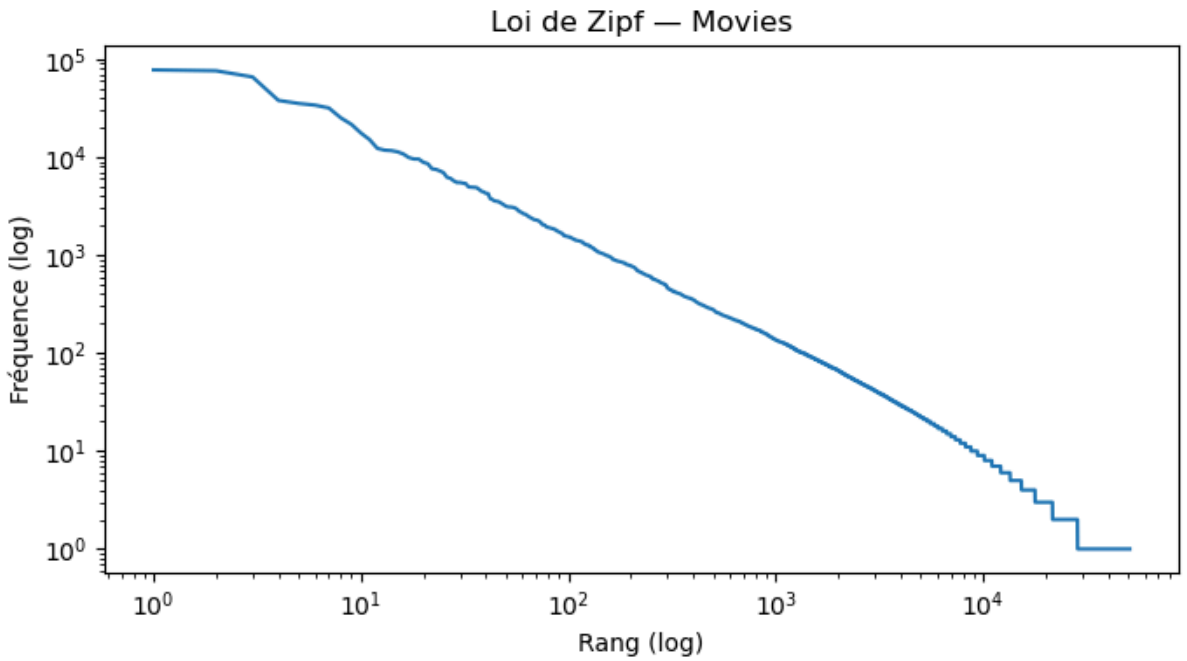
L'objectif de ce rapport est double. D'une part, il s'agit d'adopter une démarche expérimentale afin d'analyser finement l'impact des choix de prétraitement, de représentation et de modèles sur les performances, en particulier dans la partie Movie. D'autre part, la partie Président constitue une exploration des modèles de deep learning vus en cours, en particulier le fine-tuning de modèles de type Transformer (CamemBERT), avec un objectif prioritaire de performance et de généralisation.

Chaque partie suit une structure similaire : une analyse exploratoire des données (EDA) permettant de comprendre les spécificités du corpus, suivie d'une série d'expérimentations guidées par ces observations. Cette approche itérative permet de relier directement les choix méthodologiques aux résultats obtenus.

2 Movie

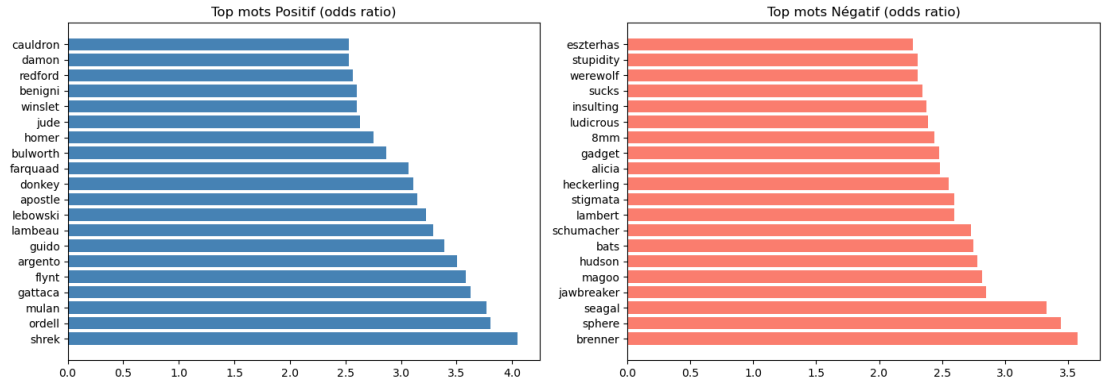
2.1 Analyse

Loi de Zipf

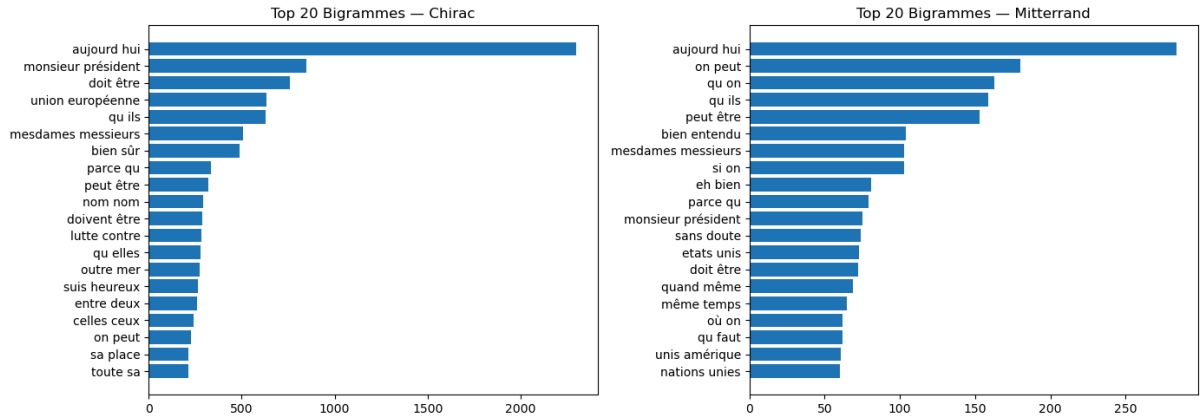


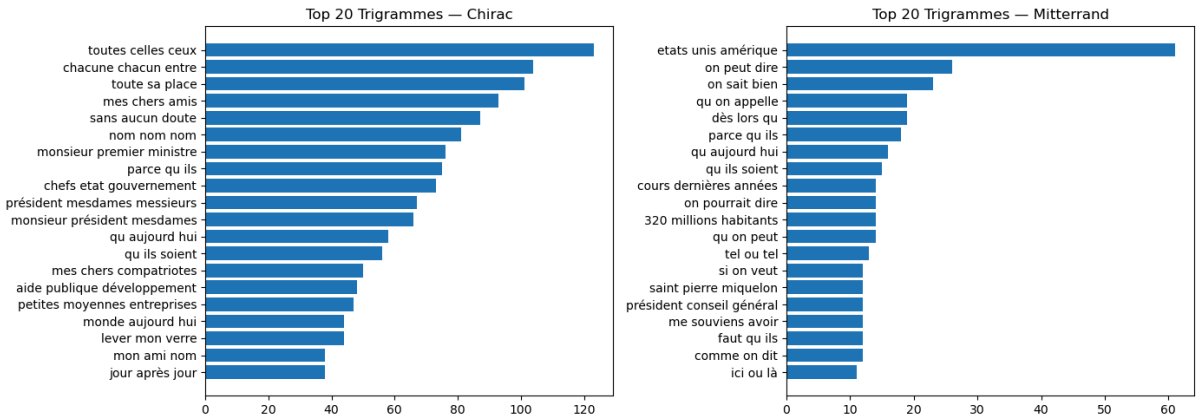
Nous sommes dans un contexte d'équilibre parfait 50/50 avec 1000 reviews dans chaque classe. Il n'y aura donc pas de problème ici de class weight ou de sur et under sampling.
Zipf montre que Movie suit bien la loi de Zipf. Le corpus est suffisamment grand et le vocabulaire suit une distribution naturelle. L'effet escalier apparaît aussi tard confirmant que le corpus est assez grand pour être statistiquement fiable. On placerait le seuil de stop word entre 10 et 50.

odds ratio

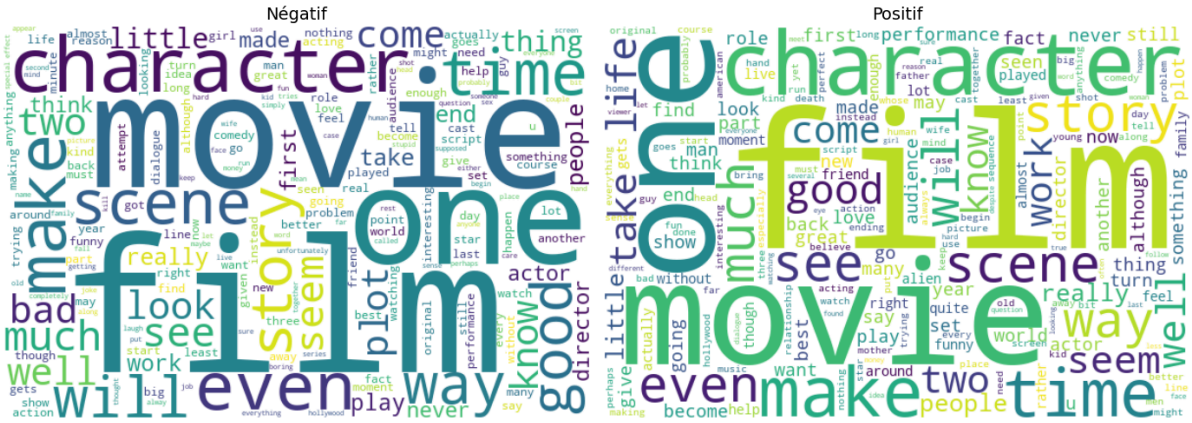


Globalement on voit apparaître des deux côtés des marqueurs de films ou réalisateurs connus. Et côté négatif on ajoute des termes marqueurs de sentiments négatifs. Et avec les bi grammes et tri gramme par classe on ne fait que confirmer notre visualisation avec les odds ratio.





Word cloud



Tout est cohérent, les termes et marqueurs positifs/négatifs sont bien associés à chaque classe. Attention à ce que le modèle n'apprenne pas indirectement que tel ou tel réalisateur ou film présent dans le corpus positif soit associé au positif et inversement.

2.2 Expérimentation

Dans un premier temps, une représentation Bag-of-Words est étudiée, en analysant l'effet de différentes stratégies de prétraitement (aucun traitement, suppression de stopwords, stemming) sur plusieurs modèles classiques : régression logistique, SVM linéaire et Naive Bayes. L'impact de l'ajout de bigrammes et de la dimensionnalité des vecteurs est également exploré.

Dans un second temps, l'étude est étendue à une représentation TF-IDF, afin d'observer les différences de comportement des modèles lorsque les mots sont pondérés en fonction de leur importance dans le corpus.

Enfin, les meilleures configurations issues de ces analyses sont sélectionnées et optimisées via une recherche d'hyperparamètres, puis évaluées sur le challenge Kaggle afin de mesurer leur capacité réelle de généralisation. Cette dernière étape permet de confronter les performances observées en validation aux résultats obtenus en conditions réelles.

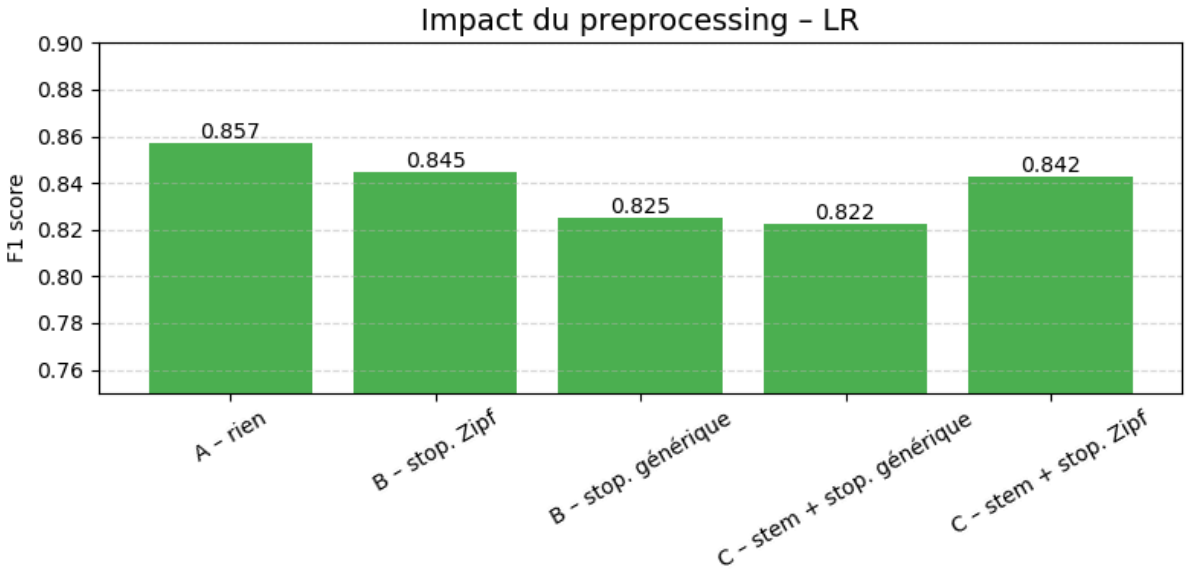
2.2.1 Protocole Bag of words

Trois configurations de prétraitement ont été testées :

1. A (aucun prétraitement, CountVectorizer brut à la main)
2. B (lowercasing + suppression des stopwords, en variant stopwords Zipf vs générique sklearn)
3. C (B + stemming Porter).

Chaque configuration est évaluée sur trois modèles : Régression Logistique (LR), SVM linéaire, et Naive Bayes (NB). La métrique principale est le F1-score macro.

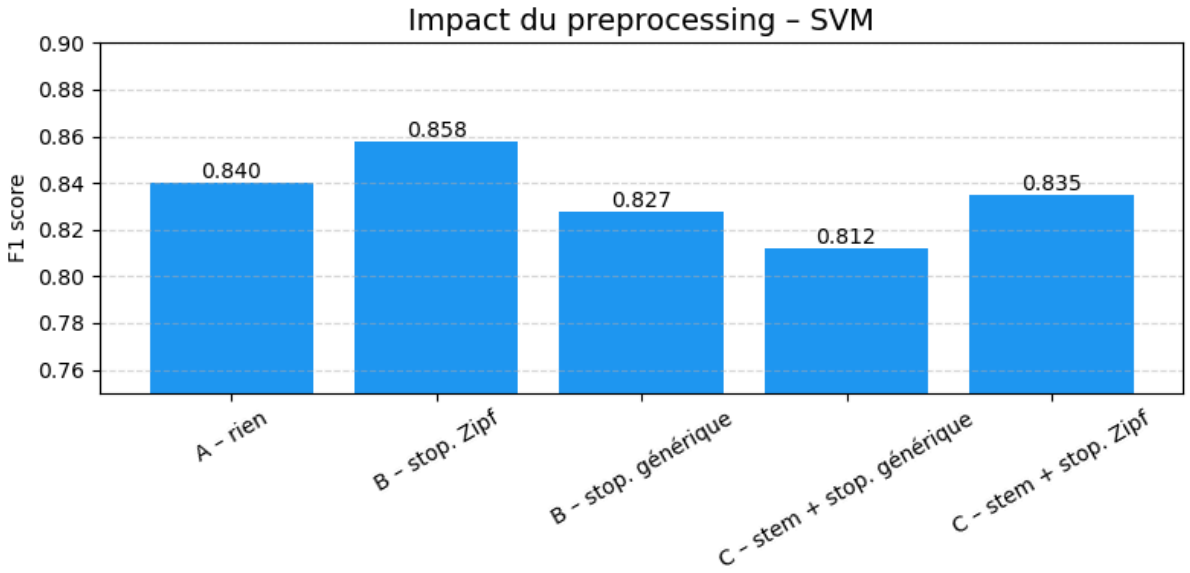
Impact du prétraitement par modèle



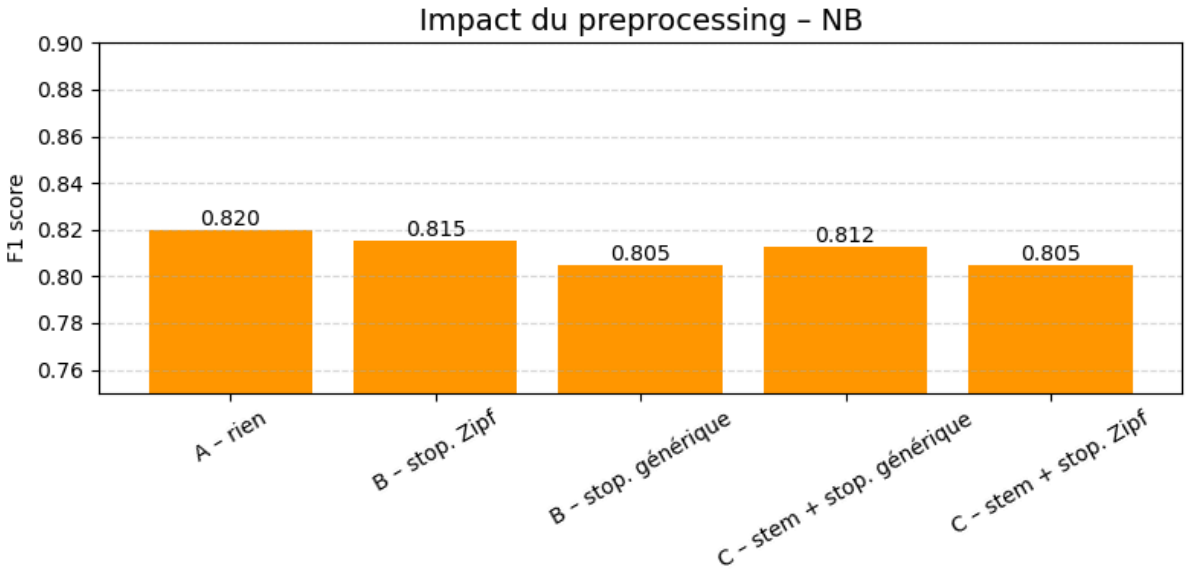
Le graphe sur LR révèle un classement clair : la configuration A (aucun prétraitement) reste la meilleure (0.857), suivie de B-Zipf (0.845), puis C-Zipf (0.842), puis B-générique (0.825), et enfin C-générique (0.822). Deux enseignements se dégagent.

D'abord, les stopwords Zipf construits sur le corpus sont systématiquement supérieurs aux stopwords génériques de sklearn. La liste sklearn est figée et conçue pour l'anglais général, elle supprime des mots qui, dans un corpus de critiques de films, peuvent porter une information sentimentale. Les stopwords Zipf eux, sont extraits directement des données et capturent les mots réellement non discriminants pour ce corpus spécifique.

Ensuite, le stemming n'apporte rien, peu importe les stopwords utilisés. B-Zipf (0.845) reste devant C-Zipf (0.842), et B-générique (0.825) reste devant C-générique (0.822). En analyse de sentiment, réduire les mots à leur racine efface des nuances importantes. Le stemming est donc une perte nette d'information dans ce contexte.



Le classement pour SVM diffère notablement de LR : cette fois c'est B-Zipf qui domine (0.858), devant A (0.840), puis C-Zipf (0.835), B-générique (0.827) et C-générique (0.812).Le point clé ici est que SVM bénéficie du nettoyage par stopwords Zipf, contrairement à LR. Cela s'explique par la nature du SVM linéaire : il cherche un hyperplan de séparation optimal et est donc plus sensible au bruit dans l'espace des features. Supprimer les mots non-discriminants avec des stopwords adaptés au corpus lui permet de mieux construire cette frontière. LR s'en sort sans prétraitement grâce à sa régularisation, SVM lui a besoin qu'on l'aide un peu.

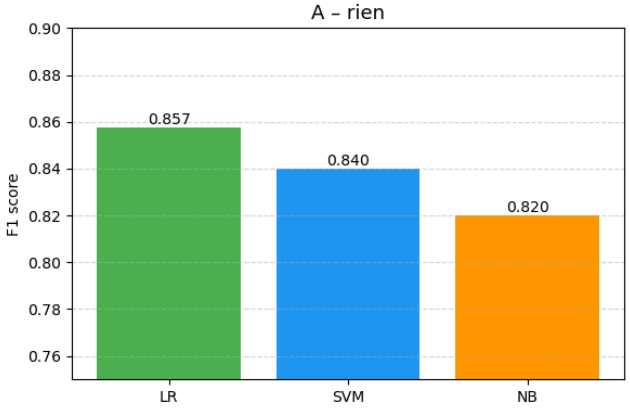


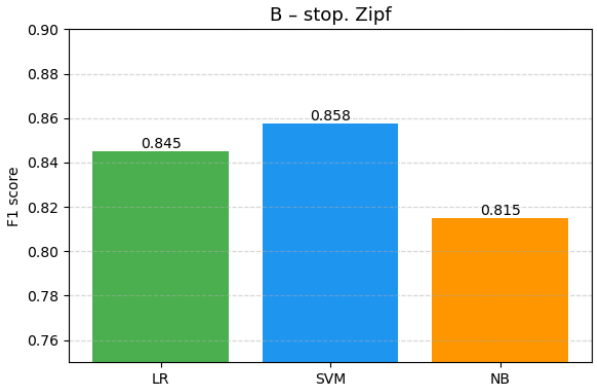
NB présente le profil le plus plat des trois modèles : les scores s'échelonnent seulement de 0.805 à 0.820, soit un écart de 1.5 points contre 3-4 points pour LR et SVM. C'est A qui reste le meilleur (0.820), et tous les prétraitements dégradent légèrement les performances.

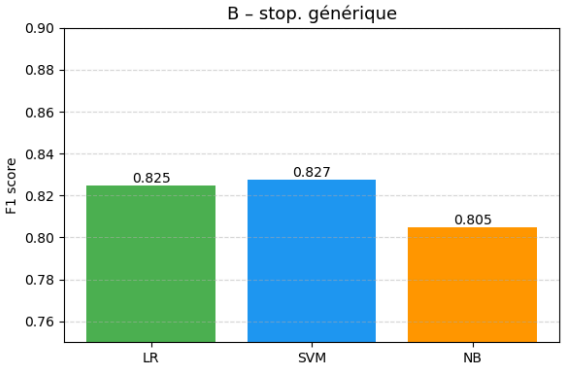
Ce comportement est caractéristique de NB : son hypothèse d'indépendance des features le rend naturellement peu sensible à la composition du vocabulaire. Qu'on lui donne 50 000 features brutes ou 30 000 features nettoyées, il calcule des probabilités indépendantes pour chaque mot ainsi le bruit supplémentaire ne l'affecte pas vraiment.

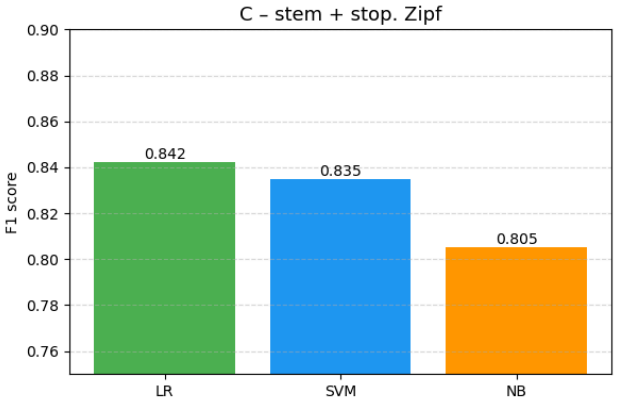
Une curiosité à noter : C-générique (0.812) dépasse B-générique (0.805), ce qui est contre-intuitif puisque le stemming dégrade partout ailleurs. La différence est trop petite pour être interprétée comme une tendance réelle.

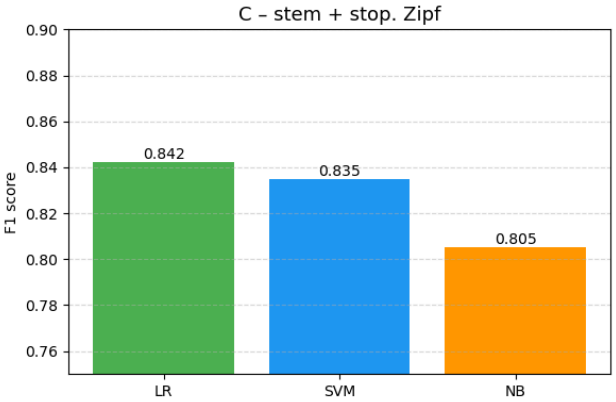
Comparaison des modèles par prétraitement











A rien : LR domine clairement (0.857 > 0.840 > 0.820). Sans prétraitement, le vocabulaire brut favorise LR dont la régularisation gère seule le bruit. SVM et NB sont en retrait.

B stop. Zipf : Le seul cas où SVM prend la tête (0.858 > 0.845 > 0.815). Un vocabulaire bien nettoyé avec des stopwords adaptés au corpus donne à SVM l'espace de features propre dont il a besoin pour construire un hyperplan efficace. LR reste proche et NB décroche :)

B stop. générique : LR et SVM sont quasi à égalité (0.825 vs 0.827), les deux en baisse significative par rapport à B-Zipf. Cela confirme que la liste sklearn nuit aux deux modèles en supprimant des mots potentiellement utiles. NB chute à 0.805.

C stem + stop. générique : La pire configuration globale. LR (0.822), SVM (0.812) et NB (0.812) sont tous au plus bas sauf pour NB. Le stemming combiné aux stopwords génériques cumule les deux sources de perte d'information. Fait notable : SVM et NB sont à égalité parfaite (ce qui traduit une dégradation si forte que les modèles convergent vers le même plancher ?).

C stem + stop. Zipf : LR reprend la tête (0.842 > 0.835 > 0.805). Les stopwords Zipf limitent les dégâts du stemming, mais on reste en dessous de B-Zipf ce qui confirmeque le stemming ne fait qu'effacer de l'information utile sans contrepartie.

Conclusion BoW

Cette série d'expériences sur le corpus Movie permet de dégager plusieurs enseignements clés.

Le premier concerne le prétraitement. Contrairement à l'intuition, plus de prétraitement ne signifie pas de meilleures performances. La configuration sans prétraitement reste compétitive, et le stemming dégrade systématiquement les résultats quel que soit le modèle. Le seul prétraitement qui apporte un gain réel est la suppression de stopwords construits sur le corpus via la loi de Zipf, la liste générique sklearn est contre-productive.

Le second concerne les modèles. LR et SVM sont tous deux sensibles à la qualité du prétraitement : LR excelle sans nettoyage grâce à sa régularisation, SVM prend l'avantage dès qu'on lui fournit un vocabulaire propre avec les stopwords Zipf. NB, lui, affiche un comportement radicalement différent : ses scores restent stables autour de 0.81-0.82 quelle que soit la configuration. C'est la marque d'un modèle “robuste” car son hypothèse d'indépendance des features l'isole des variations du vocabulaire, ce qui est à la fois sa force (fiabilité peu importe le prétraitement) et sa limite (impossibilité de progresser avec un meilleur prétraitement).

La meilleure configuration retenue est B-Zipf + SVM avec un F1 de 0.858, mais l'écart avec A + LR (0.857) est négligeable. Le vrai enseignement n'est donc pas tant quel modèle gagne, mais comment chaque modèle réagit au prétraitement : une compréhension essentielle pour la suite.

Impact de l’ajout du bigramme

Modele	Sans bigramme	Avec bigramme	delta
LR	0.857	0.860	+0.003
SVM	0.858	0.852	-0.005

Les résultats vont dans des directions opposées !

LR gagne légèrement car sa régularisation sait gérer l'explosion du nombre de features introduite par les bigrammes donc la régularisation minimise automatiquement les bigrammes rares ou non-informatifs et garde les utiles.

SVM perd car il est bien plus sensible à la malédiction de la dimensionnalité. Ajouter les bigrammes multiplie massivement la taille du vocabulaire, et dans cet espace très grand, trouver un bon hyperplan devient plus difficile. Le SVM se noie dans des features peu fréquentes qui bruitent la séparation.

Conclusion bigrammes : le gain est marginal pour LR (+0.3%) et négatif pour SVM (-0.5%). Les bigrammes ne sont pas le levier clé sur ce corpus la richesse des unigrams est déjà suffisante pour capter le sentiment. On retiendra LR + bigrammes comme légèrement meilleur mais la différence est trop faible pour être décisive.

Taille des vecteurs créés (explosion de feature avec bigramme) :

LR bag (brut) : 45751
LR bag + bigram (brut) : 442734

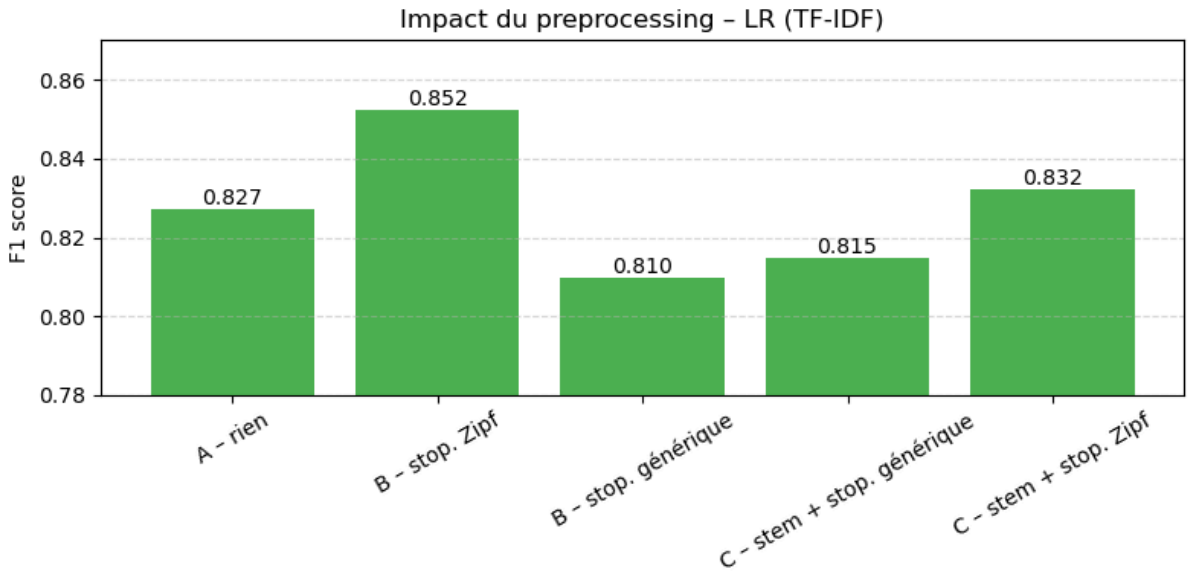
SVM bag + stop Zipf : 35647
SVM bag + stop Zipf + bi : 495141

Si on limite max_features = 100000 ou 50000

Modèles	+ bigramme 100k features	+ bigramme 50k features
SVM	0.847	0.857
LR	0.854	0.852

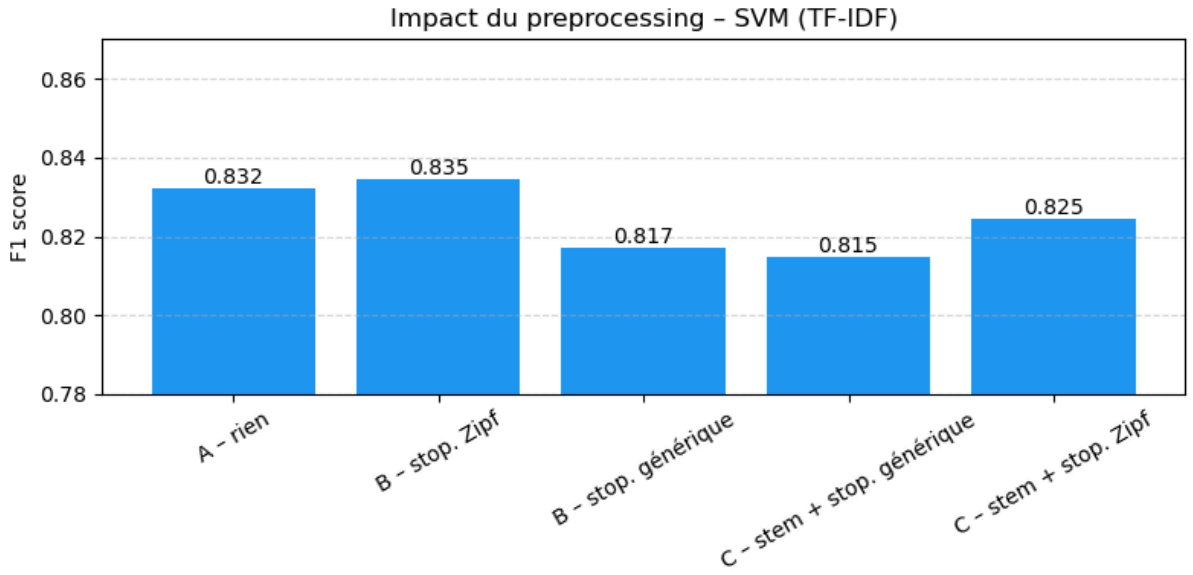
Intéressant car les deux modèles réagissent à l'inverse !
SVM préfère moins de features (50k) ce qui est cohérent avec ce qu'on sait, il souffre de la haute dimensionnalité. LR préfère plus de features (100k) grâce à sa régularisation, il gère bien les grands vocabulaires et profite des bigrammes supplémentaires. Mais dans les deux cas on reste en dessous ou à égalité avec les meilleurs scores sans bigramme (LR 0.857, SVM 0.857). Les bigrammes ne sont clairement pas le levier sur ce corpus. Les unigrammes sont assez “puissant” et expressif pour les sentiments dans le corpus.

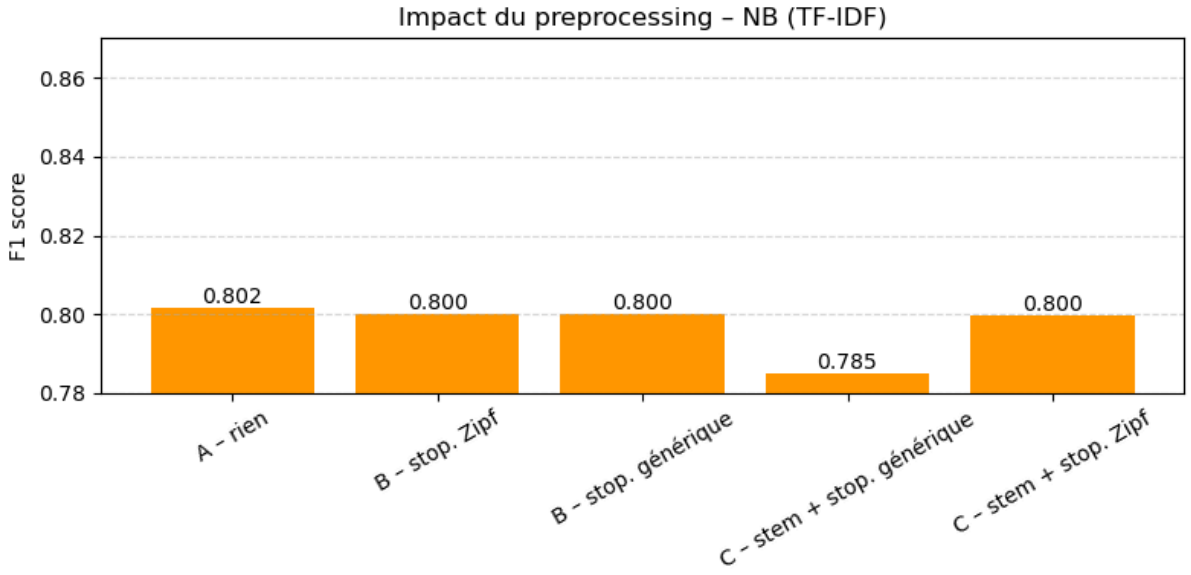
2.2.2 Protocole TF-IDF



Le graphe LR en TF-IDF révèle un classement radicalement différent du BoW : cette fois c'est B-Zipf qui domine largement (0.852), loin devant A-rien (0.827).C'est l'inversion la plus marquante par rapport au BoW où A-rien était le meilleur pour LR. En TF-IDF, ne pas faire de prétraitement pénalise le modèle : les mots ultra-fréquents comme "the", "a", "is" ont certes un IDF très faible, mais ils bruitent quand même la représentation vectorielle. Les stopwords Zipf, en les supprimant proprement, permettent à TF-IDF d'exprimer pleinement son mécanisme de pondération sur les mots vraiment informatifs.Les deux conclusions du BoW se confirment néanmoins : Zipf reste

supérieur au générique (B-Zipf 0.852 vs B-générique 0.810, écart bien plus marqué qu'en BoW), et le stemming dégrade toujours (C-Zipf 0.832 < B-Zipf 0.852).

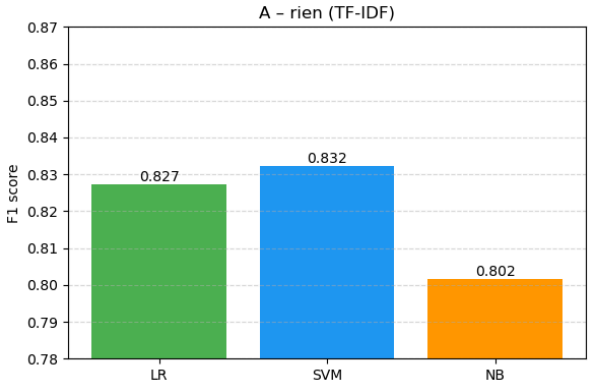


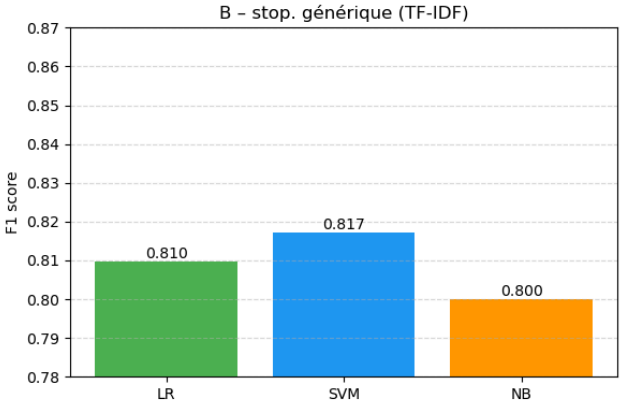


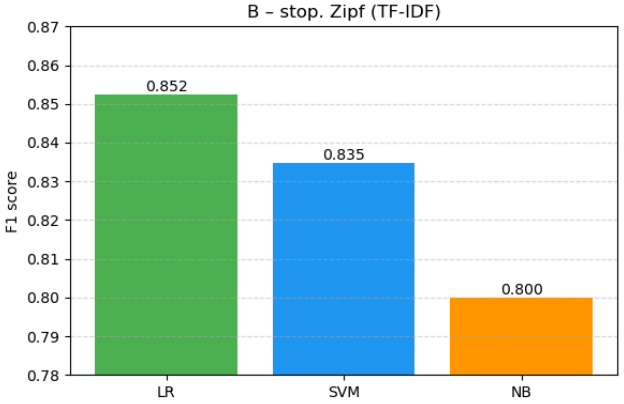
SVM en TF-IDF présente le profil le plus plat des trois modèles : les scores s'échelonnent de 0.815 à 0.835, soit seulement 2 points d'écart. Le classement est B-Zipf (0.835) > A-rien (0.832) > C-Zipf (0.825) > B-générique (0.817) > C-générique (0.815).

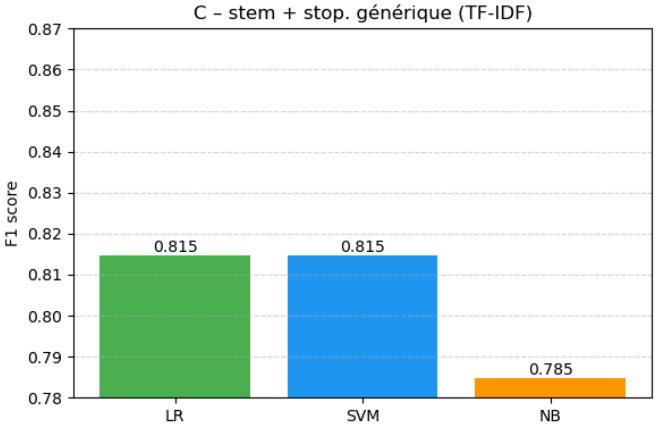
Contrairement à LR, SVM bénéficie peu du prétraitement en TF-IDF. L'écart entre B-Zipf et A-rien est minime (0.003). La pondération TF-IDF atténue déjà naturellement l'impact des mots fréquents, ce qui réduit l'utilité des stopwords pour SVM. Les conclusions habituelles tiennent néanmoins : Zipf > générique, stemming dégrade.

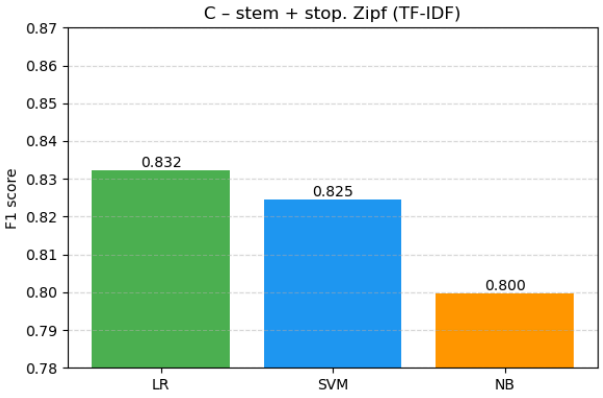
Comparaison par pretraitement











A rien : SVM prend la tête (0.832 > 0.827 > 0.802). C'est l'inverse du BoW où LR dominait sans prétraitement. TF-IDF sans nettoyage produit une représentation bruitée que SVM gère mieux grâce à sa recherche d'hyperplan -> LR est plus perturbée par les features non-informatives pondérées à tort

B stop. Zipf : LR reprend largement la tête (0.852 > 0.835 > 0.800). C'est la configuration qui révèle le mieux le potentiel de LR avec TF-IDF -> un vocabulaire propre + pondération TF-IDF donne à LR exactement ce dont il a besoin. L'écart avec SVM (0.017) est le plus grand observé jusqu'ici.

B stop. générique (TF-IDF) : SVM repasse devant (0.817 > 0.810 > 0.800). Les stopwords génériques pénalisent plus LR que SVM en TF-IDF -> en supprimant des mots potentiellement utiles au

sentiment, la pondération TF-IDF de LR perd ses meilleurs signaux. L'écart global est faible mais la tendance est claire.

C stem + stop. générique (TF-IDF) : La pire configuration, LR et SVM à égalité parfaite (0.815 = 0.815), NB s'effondre à 0.785. Même constat qu'en BoW : le cumul stemming + stopwords génériques est la combinaison la plus destructrice. L'égalité LR/SVM traduit une dégradation si forte que les deux modèles convergent vers le même plancher exactement comme en BoW C-générique.

C stem + stop. Zipf (TF-IDF) : LR domine (0.832 > 0.825 > 0.800), même classement que B-Zipf mais tout le monde a baissé. Le stemming coûte 0.020 points à LR et 0.010 à SVM par rapport à B-Zipf ce qui confirme que le stemming dégrade systématiquement quelle que soit la représentation.

Conclusion TF-IDF

La tendance la plus frappante en TF-IDF est l'inversion du leadership entre LR et SVM selon le prétraitement. SVM domine sur les configs sans nettoyage ou avec stopwords génériques (A, B-générique), LR domine dès qu'on lui fournit un vocabulaire propre via Zipf (B-Zipf, C-Zipf). NB est constant dernier à environ 0.800 quelle que soit la config

Ajout et impact des bigrammes au deux meilleur configuration tf idf

Modèle	Sans bigramme	Avec bigramme full vocab	delta
LR	0.852	0.837	-0.015
SVM	0.835	0.842	+0.007

Modele	max_features = 50k	max_features = 100k
LR	0.847	0.842
SVM	0.845	0.852

Même tendance opposée qu'en BoW ! LR souffre des bigrammes, SVM en profite avec assez de features. La différence notable en TF-IDF : SVM à 100k (0.852) réussit à égaler la meilleure référence LR sans bigramme, ce qui n'était pas le cas en BoW où les bigrammes ne permettaient pas de dépasser les meilleurs scores unigrams

2.2.3 Comparaison et tendance des modèles BoW vs TF-IDF :

Meilleur score par représentation

Modèle	Bow(meilleur)	TF-IDF(meilleur)	delta
LR	0.857(A-rien)	0.852(B-zipf)	-0.005
SVM	0.858(B-zipf)	0.835(B-zipf)	-0.023
NB	0.820(A-rien)	0.802(A-rien)	-0.018

Meilleur absolu Bow vs TF-IDF + bigrammes :

Config	Score
Bow : LR + bigramme	0.860
Bow : SVM + B-Zipf	0.858
TF-IDF : LR + B-Zipf	0.852
TF-IDF : SVM + bigramme 100k	0.852

Analyse :

BoW domine TF-IDF sur ce corpus, ce qui est contre-intuitif mais s'explique bien. TF-IDF pénalise les mots fréquents via l'IDF or en analyse de sentiment, les mots les plus fréquents sont souvent les plus utiles : "good", "bad", "great", "terrible" apparaissent dans beaucoup de reviews et ont donc un IDF faible. BoW les garde à leur pleine valeur de comptage, TF-IDF les réduits inutilement.

Les deux représentations confirment néanmoins les mêmes tendances structurelles : Zipf > générique, stemming nuit toujours, NB est robuste mais plafonné.

La différence majeure entre les deux est le comportement de LR : en BoW il excelle sans prétraitement, en TF-IDF il a besoin de stopwords Zipf pour performer. La pondération TF-IDF rend LR plus exigeant sur la qualité du vocabulaire en entrée.

Configurations retenues pour la recherche d'hyperparamètres et le challenge :

Representation	Modele	Congif	Score actuel
BoW	LR	A-rien + bigramme	0.860
BoW	SVM	B-zipf	0.858
TF-IDF	LR	B-zipf	0.852

L'idée est de garder trois configs suffisamment différentes . On a deux représentations et deux modèles pour comparer leur généralisation réelle sur le test Kaggle. Un bon score en validation ne garantit pas un bon score en généralisation, et il est possible que TF-IDF + LR généralise mieux que BoW + LR malgré un score inférieur en validation.

La prochaine étape est donc une recherche d'hyperparametre sur chacun des trois pour optimiser C (et le penalty pour LR), puis on soumet les prédictions sur le challenge et on compare les vrais scores de généralisation.

Les résultats de la recherche sont très intéressants :

Config	Meilleurs params	F1 val
LR+BoW+Bigramme	l2,C=100	0.8625
SVM+BoW+B-zipf	C=10	0.86
LR+TF-IDF+B-zipf	l2,C=1	0.8523

LR BoW + bigramme gagne (0.8625) : la régularisation L2 avec C=100 (faible régularisation) laisse LR exploiter pleinement les bigrammes. C'est le meilleur absolu.

SVM BoW remonte bien (0.860) : C=10 confirme qu'il avait besoin de moins de régularisation que le défaut C=1.

LR TF-IDF déçoit (0.852) : C=1 est le défaut, la recherche d'hyperparamètres n'a rien apporté sur cette config. TF-IDF + LR semble plafonner.

La recherche d'hyperparamètres a été effectuée sur un ensemble de validation interne représentant 20% du corpus d'entraînement (stratifié 50/50). Les C optimaux trouvés sont respectivement 100, 10 et 1.

Cependant un ensemble de validation de 200 samples reste limité pour estimer la généralisation sur 25 000 reviews. Par raisonnement théorique on peut anticiper des ajustements :

LR avec régularisation L2 a tendance à sous-régulariser sur un petit ensemble de validation donc un C plus élevé lui permet d'exploiter pleinement le vocabulaire riche d'un grand corpus de test sans trop contraindre les coefficients utiles. Je propose donc d'appliquer un facteur 10 au dessus des C trouvés en validation (100 à 1000 et 1 à 10).

SVM à l'inverse bénéficie d'une régularisation plus forte sur un espace de features large donc un C plus faible stabilise l'hyperplan et évite l'overfitting sur des features rares du train set. Je propose donc de descendre significativement le C de SVM, passant de C=10 à C=0.001.

2.2.3 Résultats Challenge

Config	Score Challenge f1
LR+BoW C=100	76.389
LR+BoW C=1000	76.334
SVM+BoW C=0.001	79.844
SVM+BoW C=10	74.616
LR+TF-IDF C=1	81.705
LR+TF-IDF C=10	82.137

LR+TF-IDF C=10 est le grand gagnant et c'est exactement celui qu'on avait prédit avec le raisonnement théorique sur la généralisation.

Les tendances sont très cohérentes avec notre analyse :

SVM confirme qu'il a besoin de forte régularisation C=0.001 (79.8%) écrase C=10 (74.6%), écart de 5 points. Notre raisonnement était juste.

LR BoW est stable mais plafonné autour de 76% quelle que soit la régularisation.Les bigrammes bruts ne généralisent pas bien sur 25k reviews.

LR TF-IDF confirme que C=10 généralise mieux que C=1 (+0.4%) donc la pondération TF-IDF combinée à peu de régularisation est la meilleure combinaison sur ce corpus pour nos modèles !

Bigrammes purs (sans unigrammes)

Pour pousser l'analyse plus loin, le meilleur modèle (LR + TF-IDF + B-Zipf) est réentraîné en utilisant uniquement les bigrammes, sans aucun unigramme.

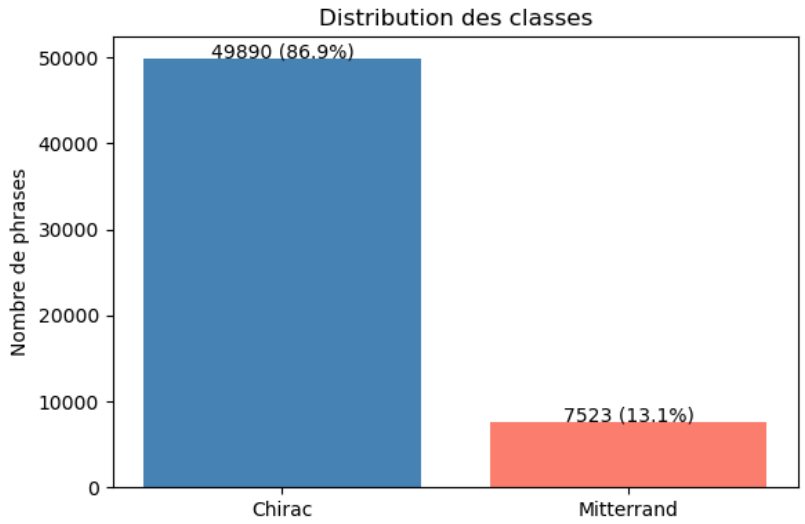
Metrique	Score
F1	79.234
Accuracy	80.392
Precision	84.207
Recall	74.816

Le résultat est remarquable : avec uniquement des paires de mots consécutifs, le modèle atteint quand même 79% de F1, soit seulement 3 points en dessous du meilleur score absolu. Cela révèle que le sentiment dans les critiques de films est encodé de manière très redondante. Les bigrammes capturent suffisamment de signal pour classer correctement 4 reviews sur 5. La suppression des unigrammes revient pourtant à retirer les mots les plus fréquents et les plus porteurs de sentiment. Le fait que le modèle survive aussi bien sans eux montre que ces signaux sont largement reconstruits par leurs contextes bigrams voisins. La precision élevée (84.2%) montre que quand le modèle dit "positif" il a raison dans 84% des cas, mais le recall plus faible (74.8%) indique qu'il manque une partie des vrais positifs. En pratique, les bigrammes seuls constituent une représentation étonnamment efficace des sentiments !

3 Président

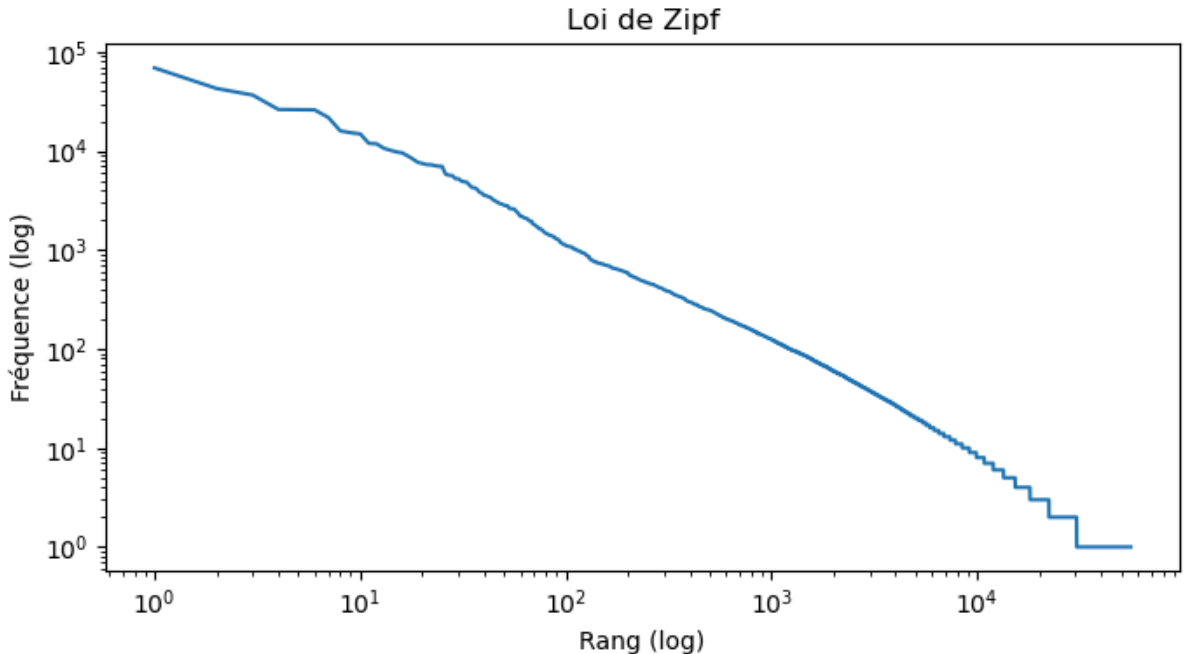
3.1 Analyse

Le corpus présente un fort déséquilibre : 86.9% des phrases sont attribuées à Chirac contre seulement 13.1% à Mitterrand. Ce déséquilibre est un défi majeur pour l'apprentissage et justifie l'utilisation de class weights et de stratégies de sous-échantillonnage ou sur-échantillonnage.

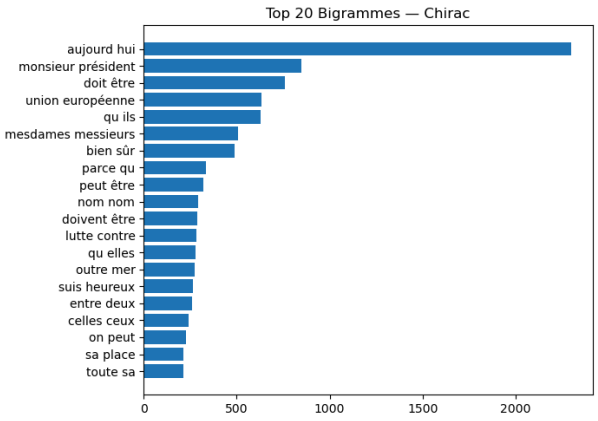


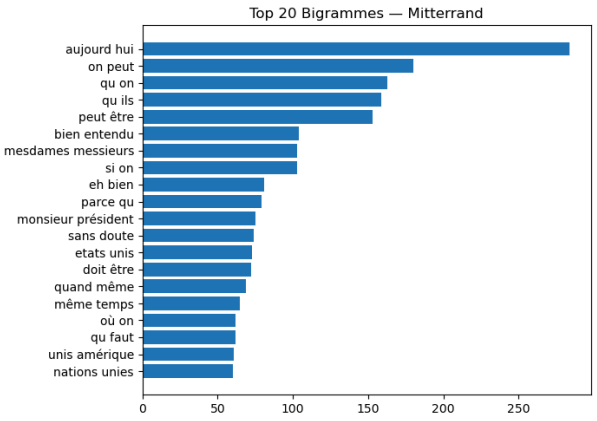
Analyse de la loi de Zipf

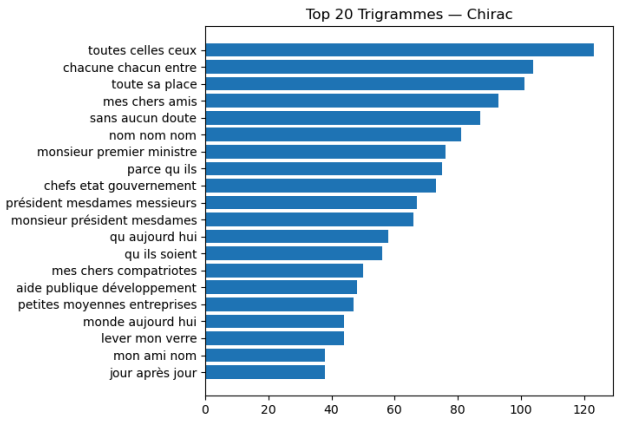
La pente de la droite montre que la loi de zipf est plutôt bien respectée donc les mots sont ni plus ni moins fréquents que prévus. On peut aussi dire que le corpus est suffisamment grand et que le vocabulaire en terme de fréquence est typiquement politique c'est à dire concentré et formel ("chute" vers 50 donc on peut considérer ça comme stopwords)

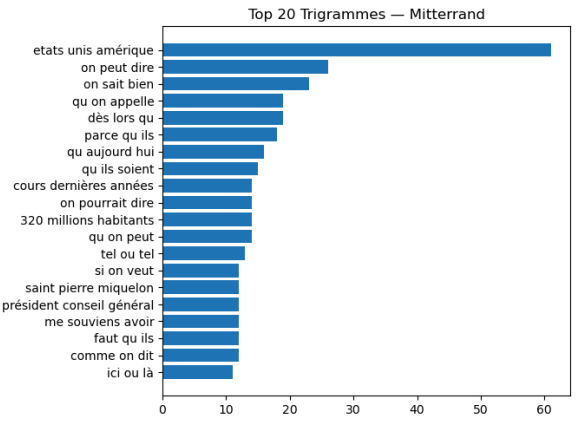


Bigrammes et Trigrammes par classe





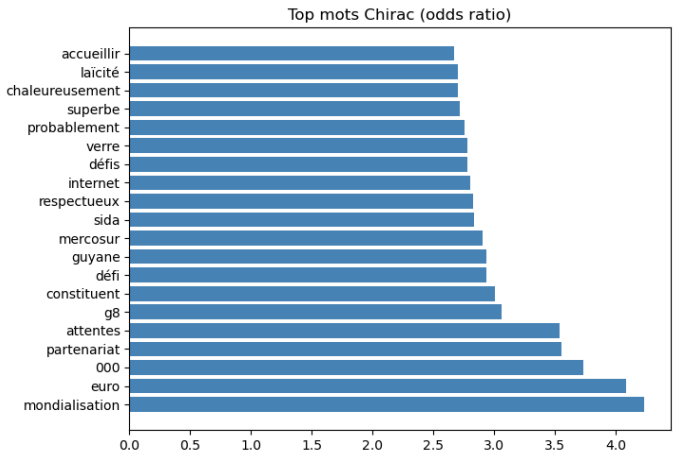


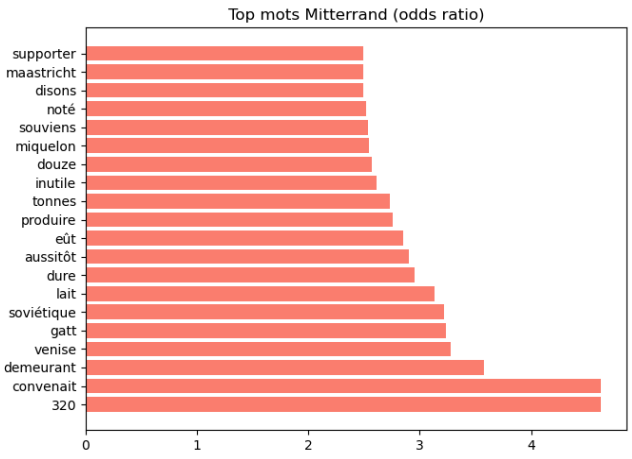


Ces bigrammes et trigrammes sont créés après suppressions des stops words avec la loi de zipfs pour éviter les “et la” ou encore “dans le” trop présent dans le corpus et ne montrant pas assez les distinctions et styles de chaque président. On peut voir un style plutôt argumentatif chez mitterrand (on peut, bien entendu…) et un style plutôt “protocolaire” pour chirak (union européenne, mesdames messieurs) dans les bigrammes. Les trigrammes montrent des formules plutôt collectives chez chirak (“mes chers amis”, “mes chers compatriotes”) et des formules “géographique” (etats unis d’amerique, saint pierre miquelon”)

En temps normal, on affiche les ngrammes sur tous le corpus mais je trouvais ca plus intéressants par classe pour que ca complète bien avec la section suivante qu’est l’odds ratio

Odds ratio





Intéressant de noter sur l'odds ratio qu'il ne capture pas seulement le "style" mais aussi l'époque. On remarque les thèmes de mondialisation et d'euro pour leur époque ainsi que le contexte de guerre froide "sovietique" de négociation "gatt" et donc globalement des débuts de l'union européen avec "maastricht". On peut y voir un biais temporel sur notre modèle, une sorte de "déséquilibre temporel".

Word cloud



Le word Cloud confirme toutes nos hypothèses et analyses depuis le début. La suppression des stopwords sera game changer voir très importante en cas de modèle utilisant des représentation vectorielle.

3.2 Experimentation

3.2.1 Baseline

Pour la partie president la soumission et la généralisation étant plus difficile au vu du problème proposés mon objectif dans cette partie ne sera pas d'évaluer l'impact du prétraitement. Je souhaite dans cette partie explorer les modèles de type transformers afin d'apprendre et d'explorer leur finetuning. Mon objectif sera donc surtout de faire de bonnes performances ici !

Pour commencer, 2 modèles simple pour évaluer la "difficulté" à prédire sur le dataset de test

Modèle	Score f1
TF-IDF 5k + SGD simple	21.206
TF-IDF 10k + SGD amelioré	16.659

Ces scores très faibles s'expliquent directement par le déséquilibre massif du corpus (86.9% Chirac / 13.1% Mitterrand) : un modèle TF-IDF classique apprend à prédire majoritairement Chirac et obtient un F1 effondré sur la classe minoritaire Mitterrand. Les TF-IDF ici ne servent que de baseline. On remarque sur la prédiction, qu'il ne font que reproduire la distribution 86/14 qu'ils ont apprise sur train. Donc on peut déduire que le test est inversement déséquilibré ! Une information utile

montrant l'intérêt et l’importance de pousser le modèle à généraliser et pas seulement à s'arrêter à une simple prédiction de sûreté.

3.2.2 Introduction à CamemBERT

CamemBERT V0 : première tentative naïve

La première implémentation de CamemBERT pose les bases mais souffre de plusieurs problèmes de conception. Le split est effectué sur des blocs chronologiques de 50 phrases shufflés ,ce qui crée un risque de fuite d'information entre train et validation puisque des phrases du même discours peuvent se retrouver dans les deux ensembles. Le contexte est ajouté de manière unilatérale (uniquement les phrases précédentes) contrairement aux versions suivantes qui prendront aussi les phrases suivantes. La boucle d'entraînement est manuelle via AdamW sans scheduler, et la sélection du meilleur modèle se fait manuellement en regardant les metrics epoch par epoch.

Les résultats sur le challenge sont catastrophiques : F1=9.99%, AUC=3.14%, AP=7.39%. Le modèle prédit 90% de Chirac sur le test -> il a simplement appris la distribution d'entraînement (86.9% Chirac) sans généraliser. Trois problèmes majeurs sont identifiés : le split par blocs fuit de l'information, le contexte unilatéral est sous-optimal, et surtout il n'y a aucune stratégie de régularisation ou de gestion du déséquilibre au niveau de la loss. Ces enseignements guident directement la V1.

CamemBERT V1 : class weights et contexte bilatéral

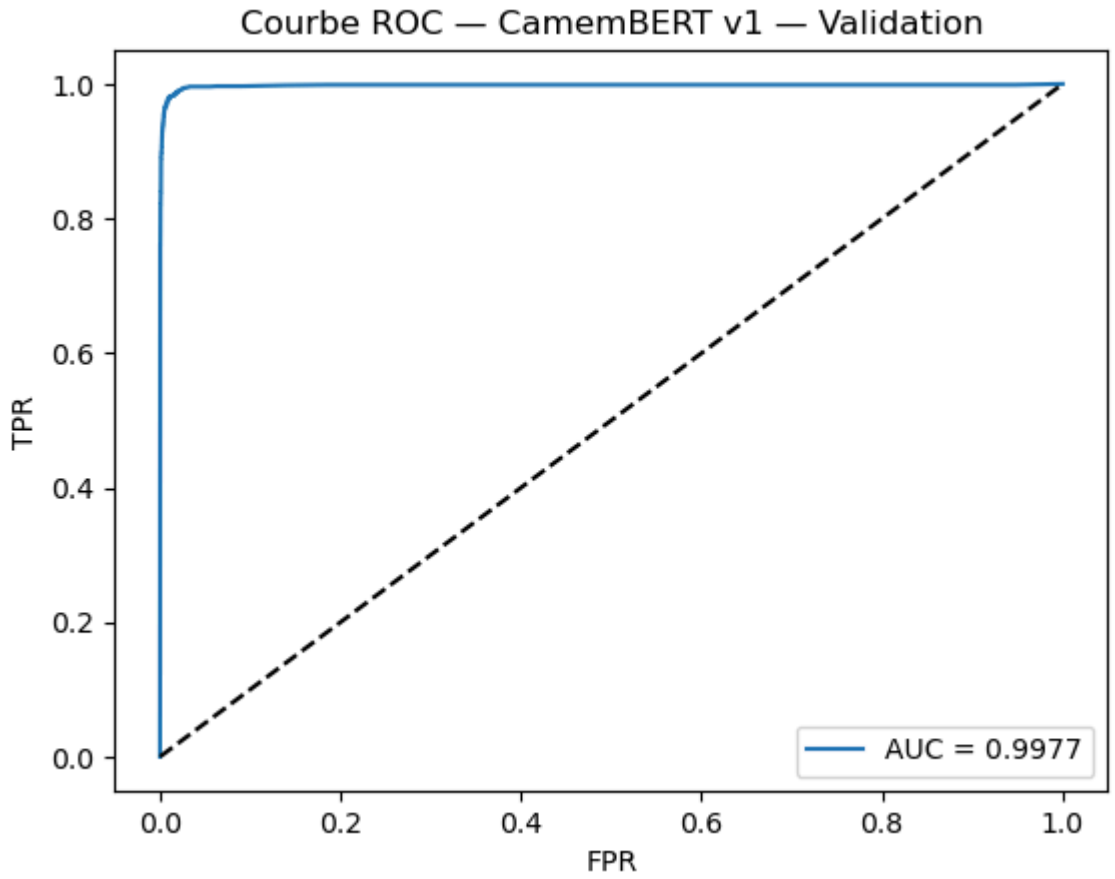
Les leçons de V0 guident directement les améliorations de V1. Trois changements majeurs sont apportés.

Le split est corrigé : on passe à un train_test_split stratifié classique (80/20) qui garantit que le déséquilibre 86.9/13.1 est préservé dans les deux ensembles et évite toute fuite d'information entre discours.

Le déséquilibre est géré via un Trainer custom qui modifie la fonction de loss avec une CrossEntropyLoss pondérée avec w_Chirac=0.575 et w_Mitterrand=3.816. Le modèle est ainsi pénalisé davantage quand il se trompe sur Mitterrand.

Le contexte devient bilatéral : chaque phrase est concaténée avec ses 2 voisines gauche et droite du même discours au lieu de seulement les phrases précédentes comme en V0.

L'entraînement utilise le Trainer HuggingFace avec scheduler warmup, weight decay à 0.01, fp16, et sélection automatique du meilleur checkpoint sur l'average precision.



F1 validation = 0.917 | Accuracy = 0.980 | AUC = 0.998

Les résultats en validation sont encourageants mais deux faiblesses subsistent. D'abord un seul split ne donne pas une estimation robuste des performances. Ensuite l'absence d'early stopping laisse le modèle overfitter en fin d'entraînement : le F1 epoch 2 (0.938) est meilleur que epoch 3 (0.916), signe que le meilleur checkpoint n'est pas le dernier. Ces deux points guident directement la V2.

CamemBERT V2 : KFold, class weights et contexte bilatéral

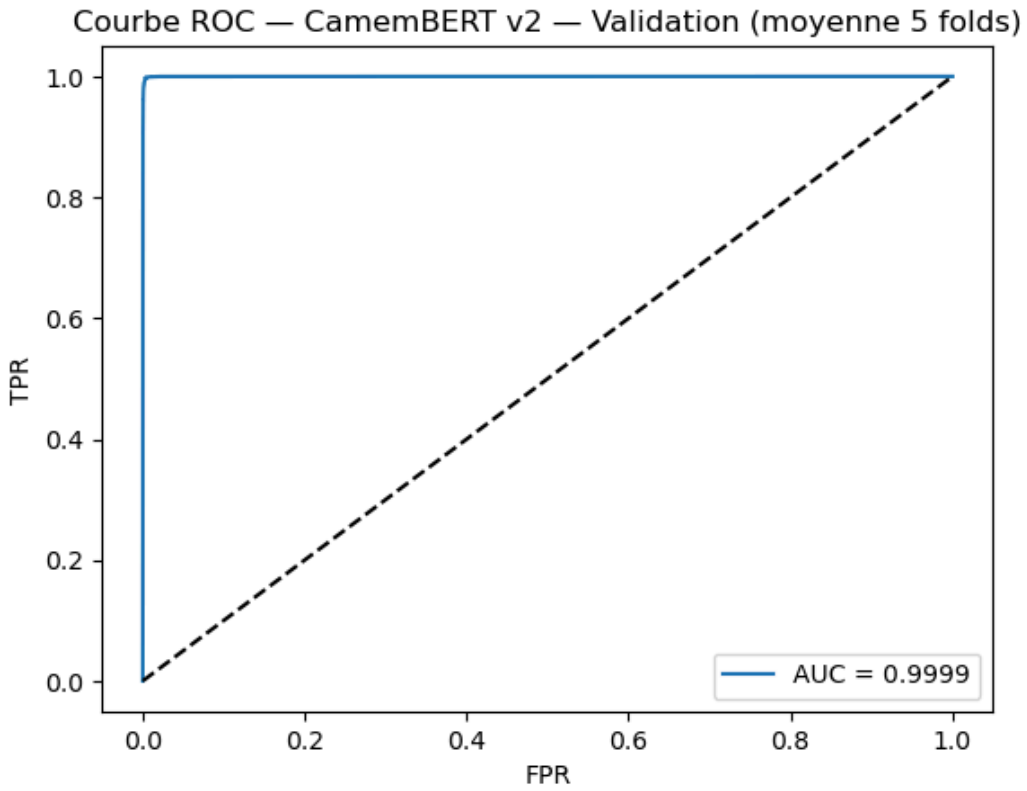
Les leçons de V1 guident directement les améliorations de V2. Trois changements majeurs sont apportés.

La robustesse statistique est assurée par une validation croisée stratifiée à 5 folds avec StratifiedKFold. À chaque fold, 80% des phrases servent à l'entraînement et 20% à la validation, en préservant le déséquilibre 86.9/13.1 dans chaque partition. Les 5 modèles entraînés prédisent ensuite indépendamment sur le test et leurs probabilités sont moyennées pour produire la prédiction finale. Ce principe de bagging réduit la variance et stabilise les prédictions.

Le déséquilibre est corrigé via un Trainer custom qui modifie comme en v1 la fonction de loss avec une CrossEntropyLoss pondérée . Ces poids sont recalculés à chaque fold proportionnellement à l'inverse des fréquences de classes (les mêmes partout en soit).

Le contexte reste bilatéral comme en V1, avec window=2 et max_len=256.

Les résultats par fold sur validation sont stables : F1 de 0.912 à 0.938 selon les folds, ce qui montre une bonne robustesse de l'approche.



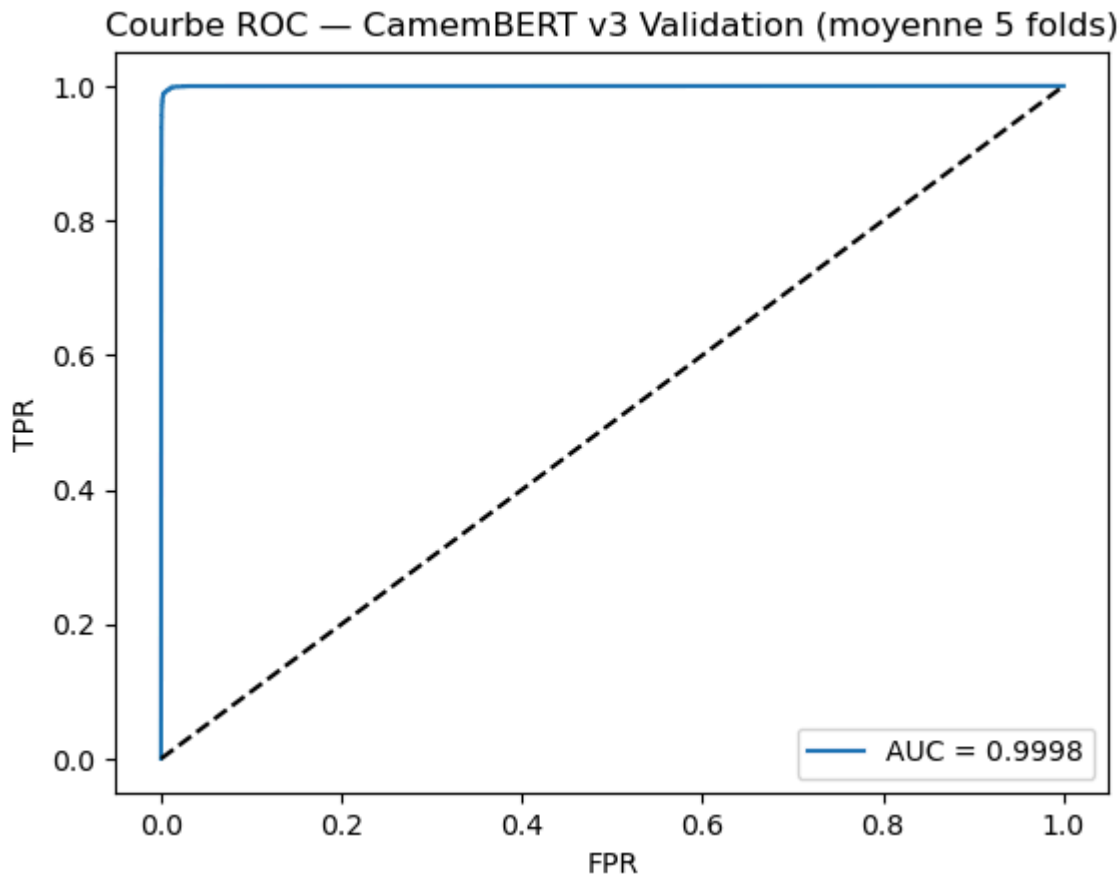
F1 validation = 0.953 | Accuracy = 0.988 | AUC = 0.9999

La progression est nette par rapport à V1 (+3.6 points de F1). Cependant en analysant les courbes d'entraînement par fold, on observe que le meilleur checkpoint n'est pas toujours la dernière epoch. Sur plusieurs folds le F1 de l'epoch 2 dépasse celui de l'epoch 3, signe d'overfitting en fin d'entraînement. Ce phénomène motive directement l'ajout d'un early stopping dans la V3.

CamemBERT V3 : V2 + early stopping

V3 est identique à V2 à un seul ajout près : un EarlyStoppingCallback avec patience=3. L'évaluation est effectuée 3 fois par epoch au lieu d'une seule, ce qui permet de détecter plus finement le moment où le modèle commence à overfitter. Concrètement, si le F1 de validation ne s'améliore pas pendant 3 évaluations consécutives, l'entraînement s'arrête automatiquement et le meilleur checkpoint est gardé.

La métrique de sélection passe également de l'average precision au F1 directement, pour être cohérent avec la métrique du challenge. Tout le reste reste identique à V2 : !



F1 validation = 0.970 | Accuracy = 0.992 | AUC = 0.9998

Le gain est net : +1.7 points de F1 en validation par rapport à V2, uniquement grâce à l'early stopping. C'est le levier le plus efficace de toute la progression. Le contexte window=2 semble néanmoins limiter l'information disponible par phrase, ce qui motive l'exploration d'une fenêtre plus large dans la V4.

CamemBERT V4 : plus de contexte !

V4 apporte deux modifications par rapport à V3. La fenêtre de contexte passe de window=2 à window=3, ce qui signifie que chaque phrase est concaténée avec jusqu'à 3 voisines gauche et droite, soit jusqu'à 7 phrases par entrée. Le max_len passe de 256 à 384 tokens pour que ce contexte supplémentaire soit réellement traité par le modèle sans soucis car à 256 tokens, une partie des phrases voisines était systématiquement coupée. L'early stopping, le KFold à 5 folds, le Trainer et tous les autres hyperparamètres restent identiques à V3.

Courbe ROC pareille que les 3 précédentes.

F1 validation = 0.968 | Accuracy = 0.992 | AUC = 0.9998

Le contexte plus large n'apporte pas de gain par rapport à V3 (-0.2 points de F1). Les séquences plus longues sont plus difficiles à optimiser sur 3 epochs, et l'information stylistique pertinente pour

distinguer les deux présidents semble déjà suffisamment captée dans une fenêtre de 5 phrases. V3 reste donc la meilleure configuration sur validation.

Lissage gaussien

Le sujet suggère d'exploiter la continuité temporelle des discours via un lissage gaussien des probabilités. L'intuition est simple : les phrases d'un même discours viennent souvent du même locuteur, donc lisser les prédictions de phrases voisines devrait régulariser les décisions isolées et réduire le bruit.

Le sigma optimal est recherché sur les prédictions ensemble sur la validation complète de chaque version.

Version	F1 sans lissage	Sigma optimal	F1 avec lissage
V2	0.953	2.0	0.961
V3	0.970	2.0	0.973
V4	0.968	2.5	0.968

Le lissage améliore V2 et V3 de manière visible (+0.8 pts et +0.3 pts) mais n'apporte rien à V4 (0.000 pts). Ce résultat est cohérent : V4 intègre déjà un contexte window=3 dans l'entrée du modèle, soit 7 phrases concaténées. Le lissage post-hoc devient donc redondant car le modèle a déjà "vu" les phrases voisines pendant l'inférence. Plus le contexte est large, moins le lissage a d'informations supplémentaires à apporter.

Cependant en appliquant ces sigmas sur le dataset de test, on observe que le lissage dégrade légèrement les performances (sans push sur github on remarque une diminution clair de la distribution des chiraks prédit après le lissage donc des performances forcement moin bonne sur la classe critique). Le sigma optimal sur validation ne généralise pas. Il y a un overfitting du sigma sur la distribution de validation qui n'est pas représentatif du test inversement déséquilibré. Les soumissions finales sont donc faites sans lissage.

3.2.3 Résultats Challenge

Version	F1	AUC	AP
TF-IDF 5k + SGD simple	21.206	15.322	7.908
TF-IDF 10k + SGD amélioré	16.659	12.432	7.731
CamemBERT V0	9.995	3.139	7.385

CamemBERT V1	25.988	85.211	64.173
CamemBERT V2	81.124	98.311	95.045
CamemBERT V3	82.061	98.610	94.773
CamemBERT V4	81.703	98.578	94.975

La progression de V0 à V3 raconte une histoire claire. Les baselines TF-IDF et V0 s'effondrent sur le F1 malgré des scores d'accuracy raisonnables car ils prédisent massivement une seule classe sans distinguer les deux locuteurs. V1 marque le premier vrai bond grâce à la gestion du déséquilibre et au contexte bilatéral. V2 et V3 confirment que le KFold et l'early stopping sont les deux leviers les plus efficaces pour la généralisation.

V3 est la meilleure configuration avec 82.061% de F1. V4 recule légèrement malgré un contexte plus large donc le gain du window=3 ne compense pas la difficulté supplémentaire d'optimiser des séquences plus longues sur seulement 3 epochs.

L'écart entre validation (F1 environ 0.97) et test Kaggle (F1 environ 0.82) est significatif et s'explique par le distribution shift : le modèle est entraîné sur un corpus majoritairement Chirac (86.9%) et doit généraliser sur un test majoritairement Mitterrand. C'est un problème fondamental que même CamemBERT ne résout pas complètement avec 3 epochs de finetuning.

On observe que CamemBERT V2 obtient le meilleur score en Average Precision, indiquant une meilleure qualité de classement des probabilités, bien que V3 soit supérieur en F1. Cela met en évidence un compromis entre optimisation du seuil de décision et qualité globale du ranking.

L'AUC très élevée de toutes les versions CamemBERT (98.3% à 98.6%) montre que le modèle classe correctement les probabilités relatives. Il sait distinguer Chirac de Mitterrand en termes de ranking. Le problème vient du seuil de décision à 0.5 qui n'est pas optimal sur un test inversement déséquilibré.

V0 présente un cas “pathologique” intéressant : son AUC de 3.1% est inférieur au hasard (50%), ce qui signifie que ses probabilités sont inversées donc il prédit des probabilités élevées pour la mauvaise classe. C'est clairement la signature d'un modèle qui a appris la distribution inverse sans aucune gestion du déséquilibre.

4 Conclusion

Ce projet avait pour objectif d'explorer deux tâches de classification textuelle : l'analyse de sentiment sur des critiques de films et la reconnaissance de locuteur sur des discours présidentiels. Les deux tâches, bien que proches dans leur formulation, se révèlent radicalement différentes dans leurs défis et leurs solutions optimales.

Sur la tâche Movie, les modèles classiques se comportent étonnamment bien. LR + TF-IDF + B-Zipf avec C=10 atteint 82.1% de F1 sur le challenge, et l'exploration systématique des prétraitements,

représentations et hyperparamètres a permis de comprendre finement pourquoi. L'enseignement principal est que plus de prétraitement ne signifie pas de meilleures performances, et que le bon modèle dépend de la représentation utilisée. LR et SVM ont des comportements opposés face aux bigrammes et au prétraitement, ce qui illustre l'importance de comprendre les mécanismes internes de chaque modèle avant de les appliquer.

Sur la tâche Président, les modèles classiques s'effondrent immédiatement face au déséquilibre de classes. La grande leçon de cette partie est qu'un bon score de validation peut masquer un modèle complètement inutile en généralisation. Le passage à CamemBERT avec une gestion rigoureuse du déséquilibre, un contexte bilatéral, et une validation croisée stratifiée a permis d'atteindre 82% de F1 sur le challenge. Chaque amélioration apportée à la pipeline (class weights, KFold, early stopping, contexte élargi) a été motivée par l'analyse des erreurs de la version précédente.

Le lissage gaussien, bien qu'efficace en validation, ne généralise pas sur le test. C'est une leçon importante sur l'overfitting du post-processing : optimiser un hyperparamètre sur une distribution différente de celle du test conduit inévitablement à une dégradation.

Plus globalement, ce projet illustre deux grandes notions. La première est entre la richesse du modèle et sa capacité à généraliser : CamemBERT avec 3 epochs de finetuning sur un corpus déséquilibré souffre du distribution shift important (0.97 en val vs 0.82 en test). La seconde est entre l'optimisation locale et la robustesse globale. Le meilleur modèle en validation n'est pas toujours le meilleur en généralisation, comme l'illustre le cas LR TF-IDF C=10 qui surpasse en généralisation tous les modèles pourtant meilleurs en validation.